

Chi – quando ServeMux não basta

DIM0547 – Sprint 1 (16/04)

Prof. Fernando · UFRN · 2026.1

Acabamos de resolver o ex01 com `net/http` puro. Agora vamos ver o que muda com Chi e por quê.

0 que é Chi?

- **Lib pequena** (~3 mil linhas, 1 dependência)
- **Sobre** `net/http`: usa `http.Handler` da `stdlib`
- **Foco em uma coisa**: roteamento expressivo
- **Sem mágica**: o que você lê é o que executa

```
go get github.com/go-chi/chi/v5
```

Não é um framework. Você não "entra no mundo do Chi" — você **adiciona** Chi a um projeto Go normal.

Por que Chi (e não Gin, Echo, Fiber)?

Aspecto	Chi	Gin/Echo/Fiber
Tipo do handler	<code>http.Handler</code> (padrão Go)	<code>gin.Context</code> (lock-in)
Middleware	<code>func(http.Handler) http.Handler</code>	API própria
Compatível com libs Go	✅ Tudo funciona	❌ Precisa wrapper
Curva de aprendizado	Baixa (é HTTP)	Média (é o framework)
Performance	Suficiente	Levemente mais rápido

Critério da escolha: o que você aprende em Chi **transfere** para qualquer projeto Go. Em Gin, fica restrito ao Gin.

Hello, Chi

```
package main

import (
    "net/http"
    "github.com/go-chi/chi/v5"
)

func main() {
    r := chi.NewRouter()

    r.Get("/hello", func(w http.ResponseWriter, r *http.Request) {
        w.Write([]byte("Olá, mundo!"))
    })

    http.ListenAndServe(":8080", r)
}
```

Note: o handler é `func(w http.ResponseWriter, r *http.Request)` — exatamente igual ao da stdlib. **Zero abstração nova.**

Antes e depois: ServeMux vs Chi

ServeMux (Go 1.22+)

```
mux := http.NewServeMux()

mux.HandleFunc(
    "GET /contacts",
    listContacts,
)

mux.HandleFunc(
    "GET /contacts/{id}",
    getContact,
)

// id := r.PathValue("id")
```

Chi

```
r := chi.NewRouter()

r.Get("/contacts", listContacts)
r.Get("/contacts/{id}", getContact)

// id := chi.URLParam(r, "id")
```

Diferenças mínimas até aqui. Chi começa a ganhar quando o app cresce.

Onde Chi ajuda: grupos de rotas

```
r := chi.NewRouter()

r.Route("/api/v1", func(r chi.Router) {
    r.Get("/contacts", listContacts)
    r.Post("/contacts", createContact)
    r.Get("/contacts/{id}", getContact)
    r.Delete("/contacts/{id}", deleteContact)
})

r.Route("/api/v2", func(r chi.Router) {
    r.Get("/contacts", listContactsV2)
    // ...
})
```

ServeMux: você teria que repetir `/api/v1/...` em cada `HandleFunc`.

Chi: prefixo declarado uma vez, escopo claro.

Subrouters e Mount

Quando o app cresce, você quer **separar arquivos**:

```
// internal/api/contacts.go
func ContactsRouter() chi.Router {
    r := chi.NewRouter()
    r.Get("/", listContacts)
    r.Post("/", createContact)
    r.Get("/{id}", getContact)
    return r
}

// main.go
r := chi.NewRouter()
r.Mount("/api/v1/contacts", contacts.ContactsRouter())
r.Mount("/api/v1/orders", orders.OrdersRouter())
```

Cada módulo expõe seu próprio router. O `main` só compõe.

Middleware: o padrão universal de Go

```
func Logger(next http.Handler) http.Handler {  
    return http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {  
        start := time.Now()  
        next.ServeHTTP(w, r)  
        log.Printf("%s %s %s", r.Method, r.URL.Path, time.Since(start))  
    })  
}
```

O **padrão**: uma função que **recebe** `http.Handler` e **retorna** `http.Handler`.

Esse padrão **não é do Chi**. É da `stdlib`. Funciona em `ServeMux` puro também.
O que `Chi` adiciona é **conveniência** para aplicar middleware a grupos.

Middleware por grupo

```
r := chi.NewRouter()
r.Use(middleware.Logger)          // todos veem

r.Get("/health", healthHandler)  // só Logger

r.Route("/api", func(r chi.Router) {
    r.Use(middleware.RequestID)  // só /api tem
    r.Get("/info", infoHandler)
})

r.Route("/admin", func(r chi.Router) {
    r.Use(authMiddleware)         // só /admin exige auth
    r.Get("/users", listUsers)
})
```

Escopo claro. Sem confundir qual middleware se aplica onde.

Middleware que vem com Chi

github.com/go-chi/chi/v5/middleware

Middleware	O que faz
Logger	Log de cada request
Recoverer	Captura panics (retorna 500 em vez de derrubar)
RequestID	Adiciona <code>X-Request-Id</code> único por request
RealIP	Detecta IP real atrás de proxies
Throttle(n)	Limita N requests simultâneos
Timeout(d)	Cancela request após duração
Compress(n)	Compressão gzip/deflate

Use `r.Use(middleware.Logger)` e ganhe logs estruturados em uma linha.

404 e 405 customizados

```
r := chi.NewRouter()

r.NotFound(func(w http.ResponseWriter, r *http.Request) {
    w.Header().Set("Content-Type", "application/json")
    w.WriteHeader(http.StatusNotFound)
    json.NewEncoder(w).Encode(map[string]string{
        "error": "rota não encontrada",
    })
})

r.MethodNotAllowed(func(w http.ResponseWriter, r *http.Request) {
    w.WriteHeader(http.StatusMethodNotAllowed)
    json.NewEncoder(w).Encode(map[string]string{
        "error": "método não permitido para esta rota",
    })
})
```

Em ServeMux puro: você teria que escrever um middleware que intercepta isso.

Resumo: ServeMux vs Chi

Caso de uso	ServeMux	Chi
API pequena (<10 endpoints)	✓ Suficiente	✓ Confortável
Versionamento <code>/api/v1</code>	😞 Repetitivo	✓ Natural
Middleware por grupo	✗ Manual	✓ Built-in
404/405 customizado	😞 Wrapper	✓ Built-in
Subrouters em arquivos separados	😞 Manual	✓ <code>r.Mount</code>
Compatível com middleware de terceiros	✓	✓

Regra prática: se o seu projeto vai crescer (e o seu **vai** crescer), comece com Chi.

Para quem vem de Spring Boot

Spring Boot	Chi
<code>@RestController</code>	função que retorna <code>chi.Router</code>
<code>@RequestMapping("/api")</code>	<code>r.Route("/api", ...)</code>
<code>@GetMapping("/{id}")</code>	<code>r.Get("/{id}", handler)</code>
<code>@PathVariable</code>	<code>chi.URLParam(r, "id")</code>
<code>@RequestBody</code>	<code>json.NewDecoder(r.Body).Decode(&req)</code>
<code>@Valid</code>	<code>validator.Struct(&req)</code> (lib externa)
<code>Filter</code> / <code>Interceptor</code>	<code>func(http.Handler) http.Handler</code>
<code>@Autowired</code>	injeção manual no construtor

Os conceitos são os mesmos. A sintaxe é mais explícita.

Próximos passos

Agora vou refatorar **ao vivo** o ex01 que acabamos de fazer, migrando para Chi. Vocês vão ver:

1.  Quanto código fica idêntico (handlers, storage, JSON)
2.  Quanto fica mais simples (router setup)
3.  Como adicionar middleware embutido em 1 linha
4.  Como agrupar tudo em `/api/v1`

Depois apresento a **Lista 3** que cobre exatamente esses tópicos + validação.

Referências

Chi

- [go-chi/chi no GitHub](#)
- [Documentação oficial](#)
- [Middleware embutido](#)

Padrões de middleware em Go

- [Alex Edwards — Making and Using HTTP Middleware](#)
- [Three Dots Labs — Common anti-patterns in Go web applications](#)

Comparações

- [Awesome Go HTTP routers — listagem completa](#)
- [Why Go's net/http is enough for most apps](#)